

```

"""
ゴールデングロス スクリーナー (スマホ対応のWeb版)
起動: streamlit run app.py
"""

import pandas as pd
import streamlit as st

st.set_page_config(page_title="ゴールデングロス検出", page_icon="📈", layout="center")

SAMPLE = """7203,トヨタ自動車
6758,ソニーグループ
9984,ソフトバンクグループ
8306,三菱UFJフィナンシャル
6861,キーエンス
9432,NTT
8035,東京エレクトロン
6501,日立製作所
7974,任天堂
4063,信越化学工業
6098,リクルートHD
9983,ファーストリテイリング
8316,三井住友フィナンシャル
4502,武田薬品工業
6902,デンソー
7267,本田技研工業
8058,三菱商事
2914,日本たばこ産業
6367,ダイキン工業
4519,中外製薬"""

# ----- 計算部分
def rsi_wilder(close, n=14):
    delta = close.diff()
    up = delta.clip(lower=0).ewm(alpha=1 / n, adjust=False).mean()
    down = (-delta.clip(upper=0)).ewm(alpha=1 / n, adjust=False).mean()
    rs = up / down.replace(0, float("nan"))
    return 100 - 100 / (1 + rs)

def analyze(df, p):
    """条件を満たせば結果の辞書、満たさなければ None。"""
    if len(df) < max(p["long"], 75) + p["days"] + 2:

```

```

        return None
close = df["Close"]
s = close.rolling(p["short"]).mean()
l = close.rolling(p["long"]).mean()
m75 = close.rolling(75).mean()
diff = s - l

crossed = (diff > 0) & (diff.shift(1) <= 0)
recent = crossed.iloc[-p["days"]:]
if not recent.any() or diff.iloc[-1] <= 0:
    return None
cross_idx = recent[recent].index[-1]

if p["rising"] and not (l.iloc[-1] > l.iloc[-2]):
    return None
if p["above75"] and not (close.iloc[-1] > m75.iloc[-1]):
    return None
if p["perfect"] and not (s.iloc[-1] > l.iloc[-1] > m75.iloc[-1]):
    return None

vol_x = None
avg_vol = df["Volume"].rolling(20).mean().shift(1).loc[cross_idx]
if pd.notna(avg_vol) and avg_vol > 0:
    vol_x = df["Volume"].loc[cross_idx] / avg_vol
if p["vol_on"] and (vol_x is None or vol_x < p["vol_ratio"]):
    return None

value = (close * df["Volume"]).rolling(20).mean().iloc[-1] / 1e6
if p["value_on"] and not (value >= p["value_min"]):
    return None

rsi = rsi_wilder(close).iloc[-1]
if p["rsi_on"] and not (rsi <= p["rsi_max"]):
    return None

dev = (close.iloc[-1] - l.iloc[-1]) / l.iloc[-1] * 100
if p["dev_on"] and dev > p["dev_max"]:
    return None

return {
    "クロス日": cross_idx.strftime("%Y-%m-%d"),
    "終値": round(float(close.iloc[-1]), 1),
    "短期線": round(float(s.iloc[-1]), 1),
    "長期線": round(float(l.iloc[-1]), 1),
    "RSI": round(float(rsi), 1) if pd.notna(rsi) else None,
    "乖離率%": round(float(dev), 1),

```

```

        "出来高倍率": round(float(vol_x), 2) if vol_x is not None else None,
        "売買代金(百万円)": round(float(value), 1),
    }

def parse_tickers(text):
    items = []
    for line in text.splitlines():
        line = line.strip()
        if not line or line.startswith("#"):
            continue
        parts = [x.strip() for x in line.replace("\t", ",").split(",")]
        code = parts[0].upper().replace(".T", "")
        if code:
            items.append((code, parts[1] if len(parts) > 1 else ""))
    return items

@st.cache_data(ttl=3600, show_spinner=False)
def fetch_chunk(codes: tuple):
    """codes (例: ('7203', '6758')) の日足を取得。1時間はキャッシュを使う。"""
    import yfinance as yf

    tickers = [c + ".T" for c in codes]
    raw = yf.download(tickers, period="1y", interval="1d", group_by="ticker",
                      auto_adjust=True, progress=False, threads=True)

    out = {}
    for c, t in zip(codes, tickers):
        try:
            d = raw[t] if isinstance(raw.columns, pd.MultiIndex) else raw
            d = d.dropna(subset=["Close"])
            if not d.empty:
                out[c] = d
        except KeyError:
            pass
    return out

# ----- 画面
st.title("📈 ゴールデンクロス検出")
st.caption("日本株の日足から、短期線が長期線を上抜けた銘柄を探します。株価データはYahoo Finan

with st.expander("① 銘柄を入力", expanded=False):
    text = st.text_area("1行1銘柄 (「7203」または「7203, トヨタ) ", value=SAMPLE, hei

with st.expander("② クロスの条件", expanded=True):

```

```

c1, c2, c3 = st.columns(3)
short = c1.number_input("短期線 (日) ", 2, 50, 5)
long_ = c2.number_input("長期線 (日) ", 5, 200, 25)
days = c3.number_input("直近N営業日以内", 1, 20, 1)

with st.expander("③ 絞り込み (ONにしたものだけ適用) ", expanded=True):
    rising = st.checkbox("長期線が上向き", True)
    above75 = st.checkbox("株価が75日線より上", True)
    perfect = st.checkbox("パーフェクトオーダー (短期>長期>75日) ", False)
    vol_on = st.checkbox("クロス日の出来高が平均の○倍以上", True)
    vol_ratio = st.slider("出来高倍率", 0.5, 10.0, 1.5, 0.5, disabled=not vol_on)
    value_on = st.checkbox("平均売買代金が○百万円以上", True)
    value_min = st.number_input("売買代金 (百万円) ", 1, 100000, 50, 10, disabled=not value_on)
    rsi_on = st.checkbox("RSI(14)が○以下", True)
    rsi_max = st.slider("RSI上限", 30, 100, 70, disabled=not rsi_on)
    dev_on = st.checkbox("長期線との乖離が○%以下", True)
    dev_max = st.slider("乖離率上限 (%) ", 1, 50, 10, disabled=not dev_on)

if st.button("▶ スクリーニング開始", type="primary", use_container_width=True):
    items = parse_tickers(text)
    if not items:
        st.warning("銘柄コードを入力してください。")
        st.stop()
    if short >= long_:
        st.warning("短期線は長期線より小さくしてください。")
        st.stop()

    p = dict(short=int(short), long=int(long_), days=int(days), rising=rising, above75=above75,
              perfect=perfect, vol_on=vol_on, vol_ratio=vol_ratio, value_on=value_on,
              value_min=value_min, rsi_on=rsi_on, rsi_max=rsi_max, dev_on=dev_on)
    names = dict(items)
    codes = [c for c, _ in items]

    bar = st.progress(0, text="株価を取得中...")
    data = {}
    try:
        for i in range(0, len(codes), 100):
            part = tuple(codes[i:i + 100])
            data.update(fetch_chunk(part))
            bar.progress(min((i + 100) / len(codes), 1.0), text="株価を取得中...")
    except Exception as e:
        bar.empty()
        st.error(f"株価の取得に失敗しました: {e}")
        st.stop()
    bar.empty()

```

```

rows = []
for code, df in data.items():
    r = analyze(df, p)
    if r:
        rows.append({"コード": code, "銘柄名": names.get(code, ""), **r,
                     "チャート": f"https://finance.yahoo.co.jp/quote/{code}.T/"}

miss = len(codes) - len(data)
note = f"（取得できなかった銘柄: {miss}）" if miss else ""
if not rows:
    st.info(f"該当なし。対象 {len(codes)} 銘柄 {note}\n\n条件を減らすと見つかる場合が")
else:
    res = pd.DataFrame(rows).sort_values("クロス日", ascending=False)
    st.success(f"該当 {len(res)} 銘柄 / 対象 {len(codes)} 銘柄 {note}")
    st.dataframe(
        res, hide_index=True, use_container_width=True,
        column_config={"チャート": st.column_config.LinkColumn("チャート", displ
    )
    st.download_button("CSVでダウンロード", res.drop(columns="チャート").to_csv(i
                        file_name="golden_cross.csv", mime="text/csv", use_con

st.caption("投資判断の材料の一つです。利益を保証するものではありません。")

```